PROGRAMMATION WEB DYNAMIQUE

Benoit Gaudou - benoit.gaudou@ut-capitole.fr

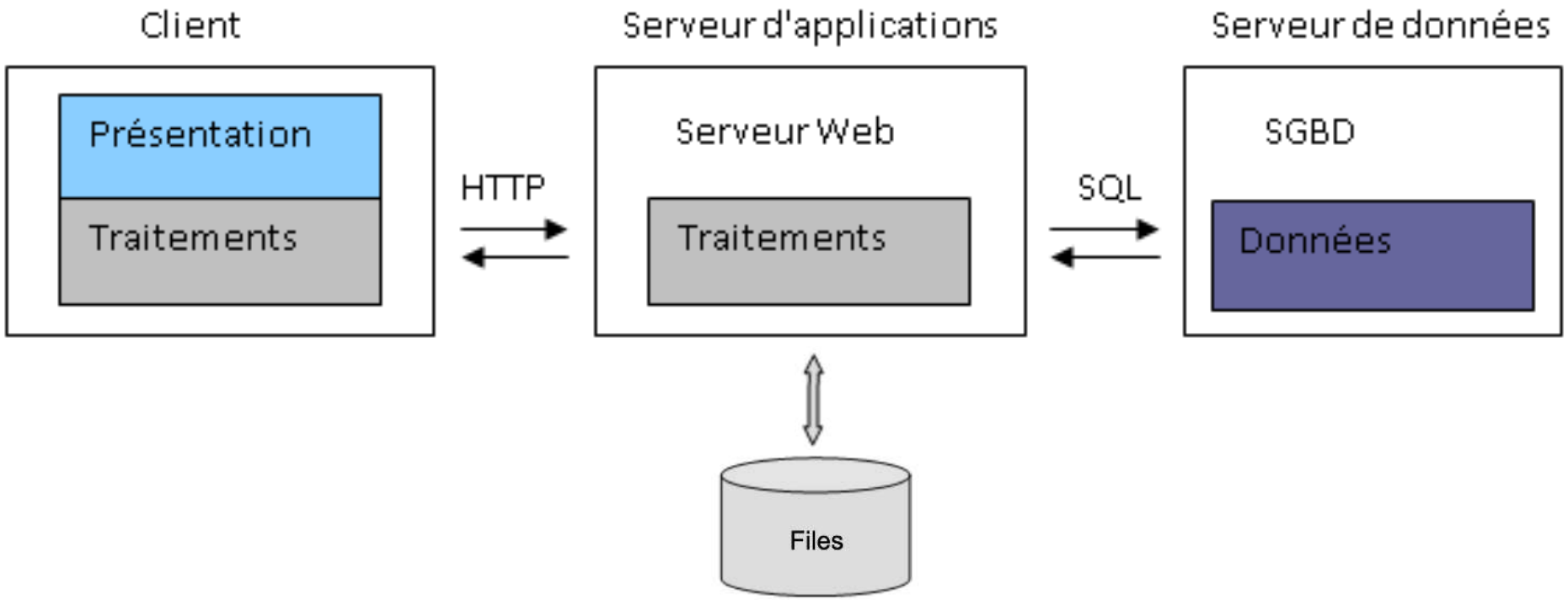
Source : Laurent Perrussel

动态网络程序设计

本诺·高杜 -benoit.gaudou@ut-capitole.fr

来源：Laurent Perrussel

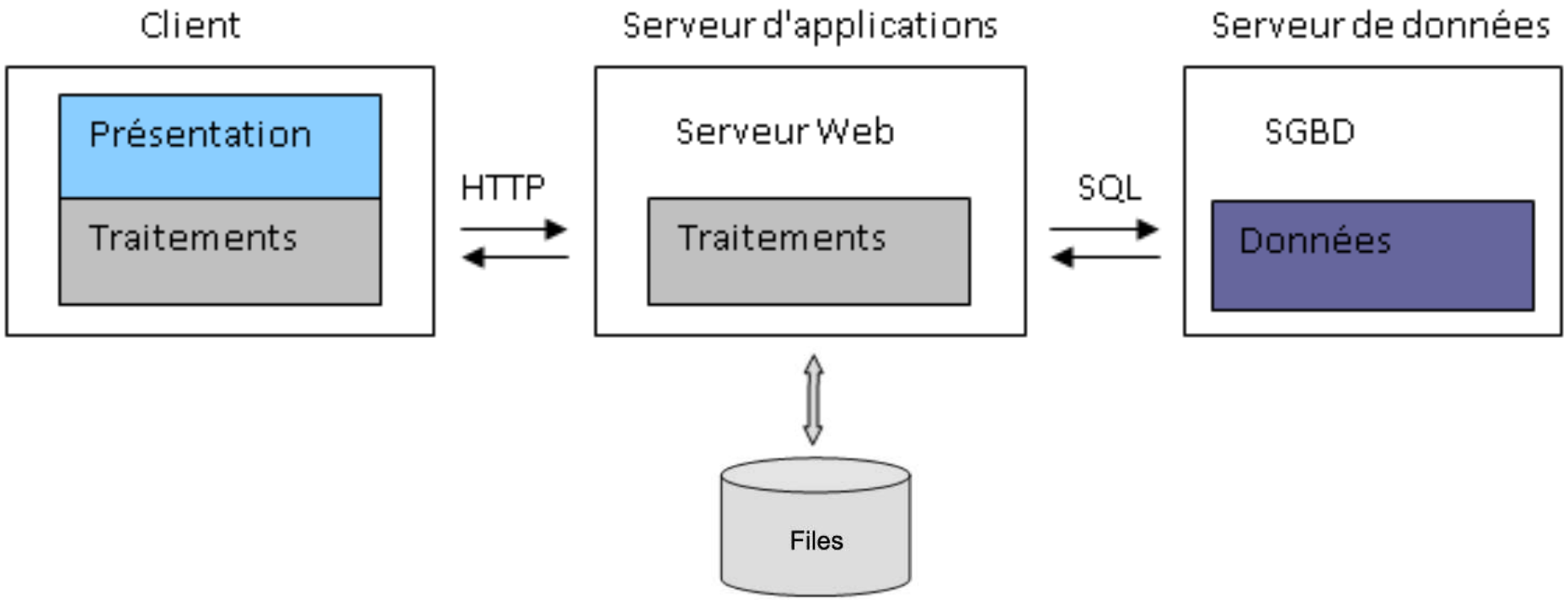
FRONTEND VS BACKEND



FrontEnd
HTML,CSS,Javascript

BackEnd
PHP, Node.JS, Python (Flask, Django),
Java (Springboot)…

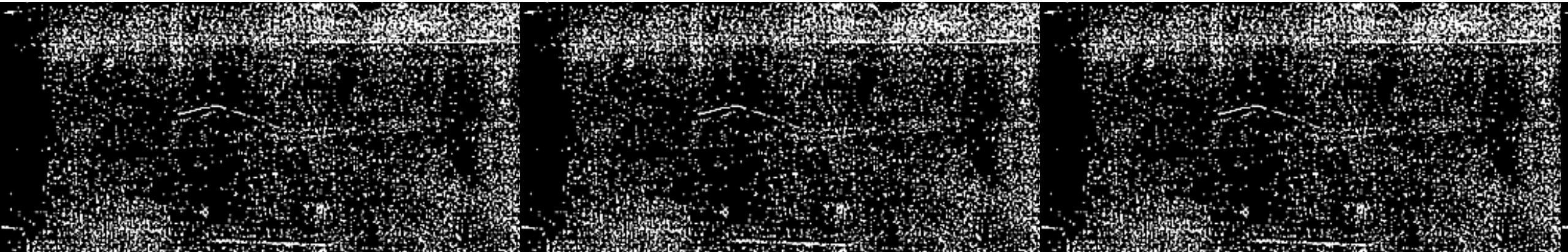
FRONTEND VS BACKEND



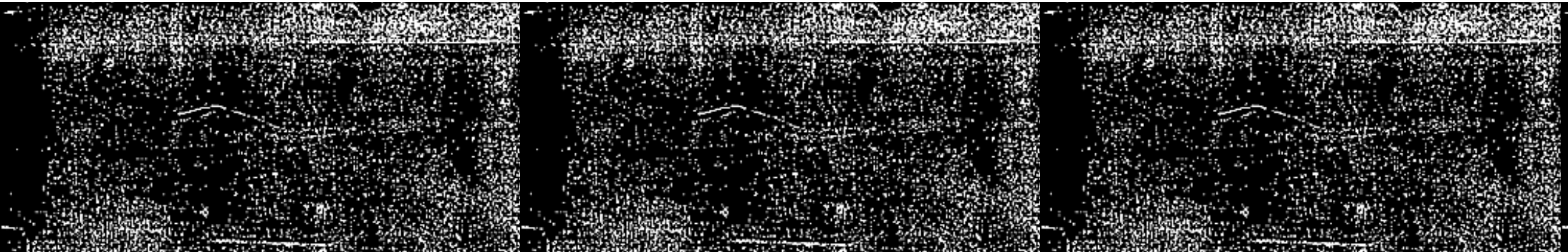
前端
HTML、CSS、JavaScript

后端
PHP、Node.JS、Python（Flask、Django）、
Java（Springboot）.....

WEB SERVICE

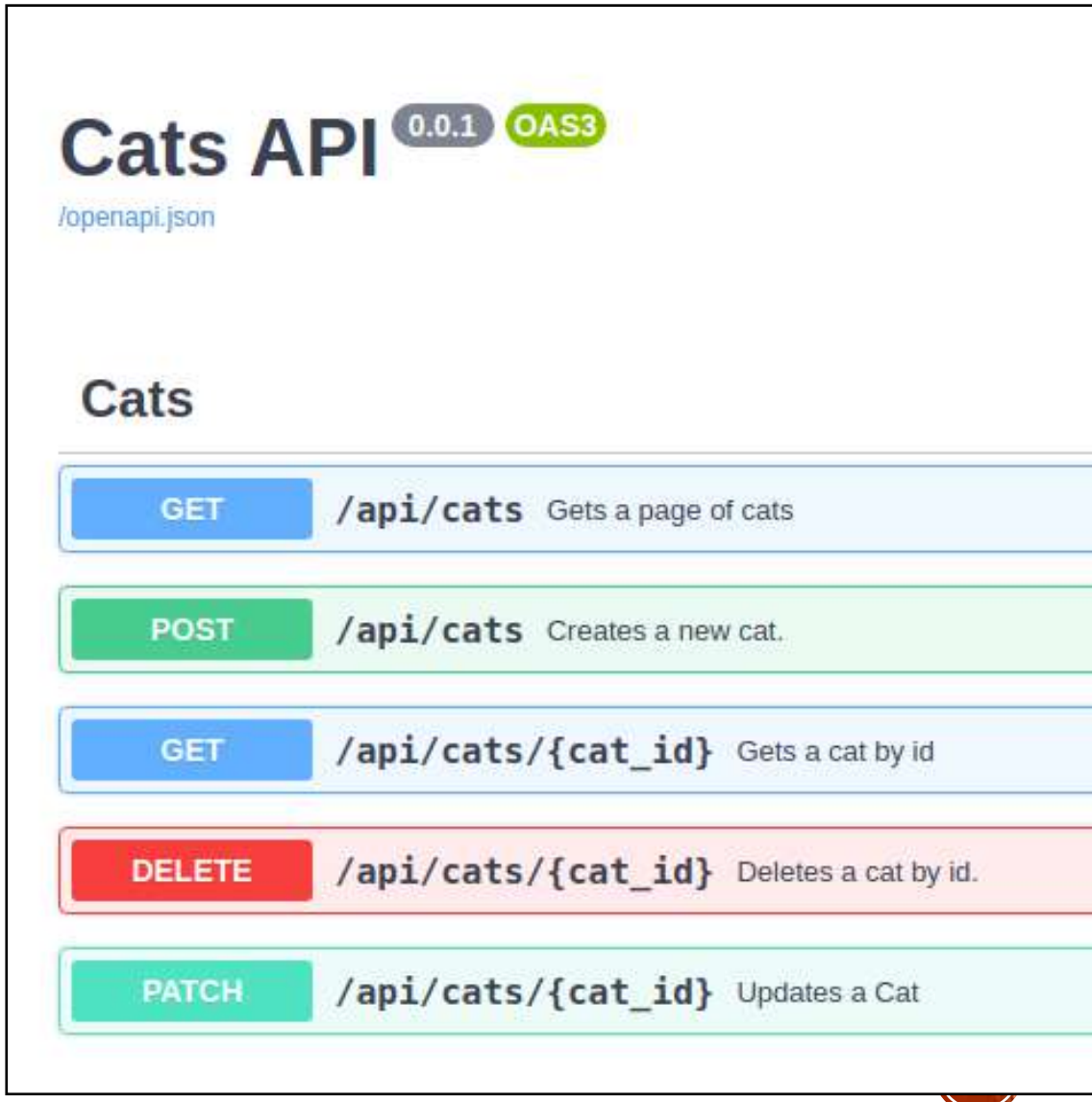


WEB SERVICE



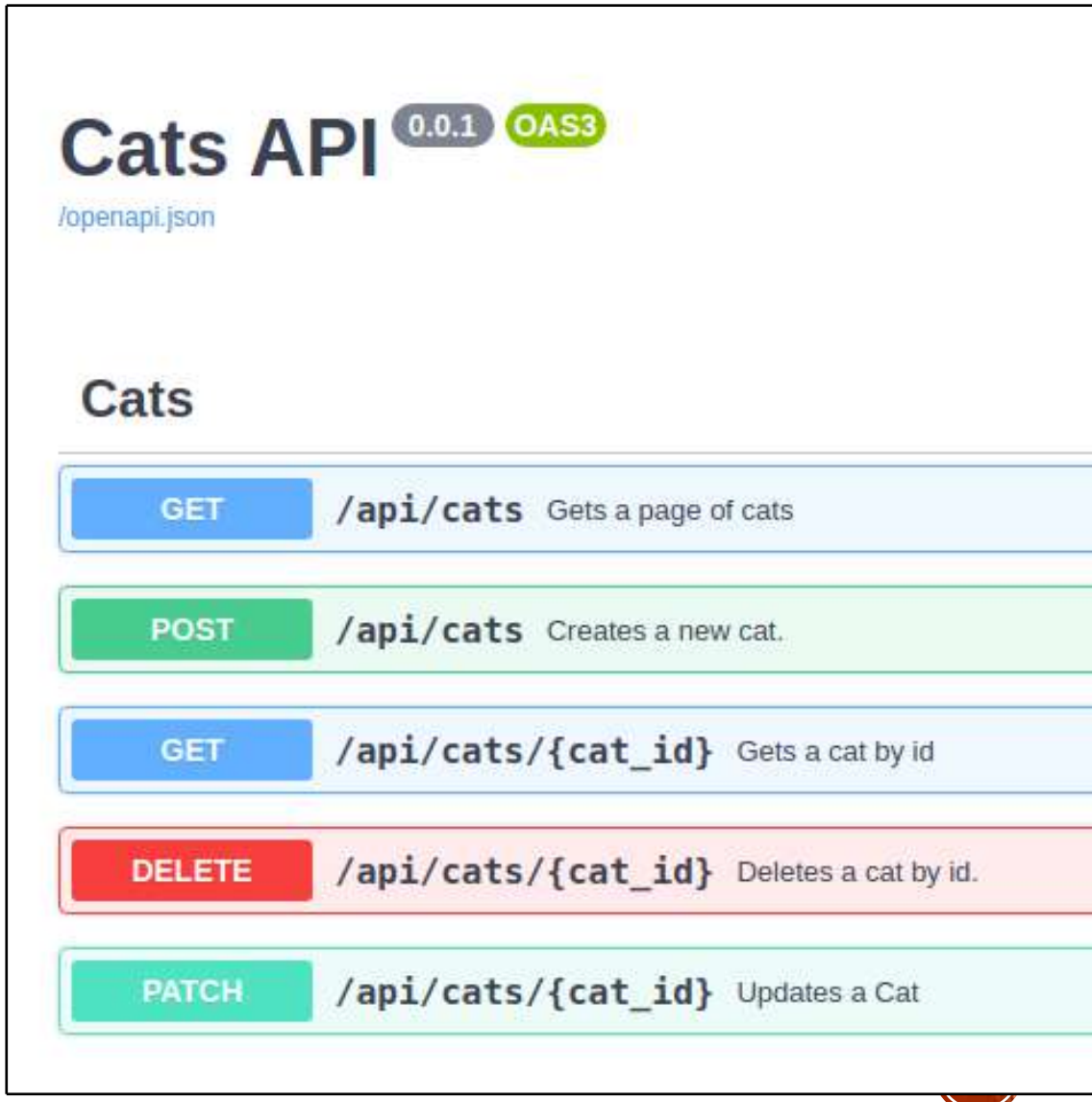
WEB SERVICES - DEFINITIONS

- Un **Web Service** est une **entité logicielle** qui **expose** des fonctionnalités au **consommateur**
  - Sur internet ou intranet
  - Au travers de **fonctions qui peuvent être appelées via internet**
- **Basé sur les standards du Web**
  - HTTP, XML, SOAP, WSDL, UDDI,…
- Peut être be implémenté dans n'importe **quel langage** et plate-forme
- Les web services sont des applications **autonomes**, **modulaires**, **distribuées** et **dynamiques**



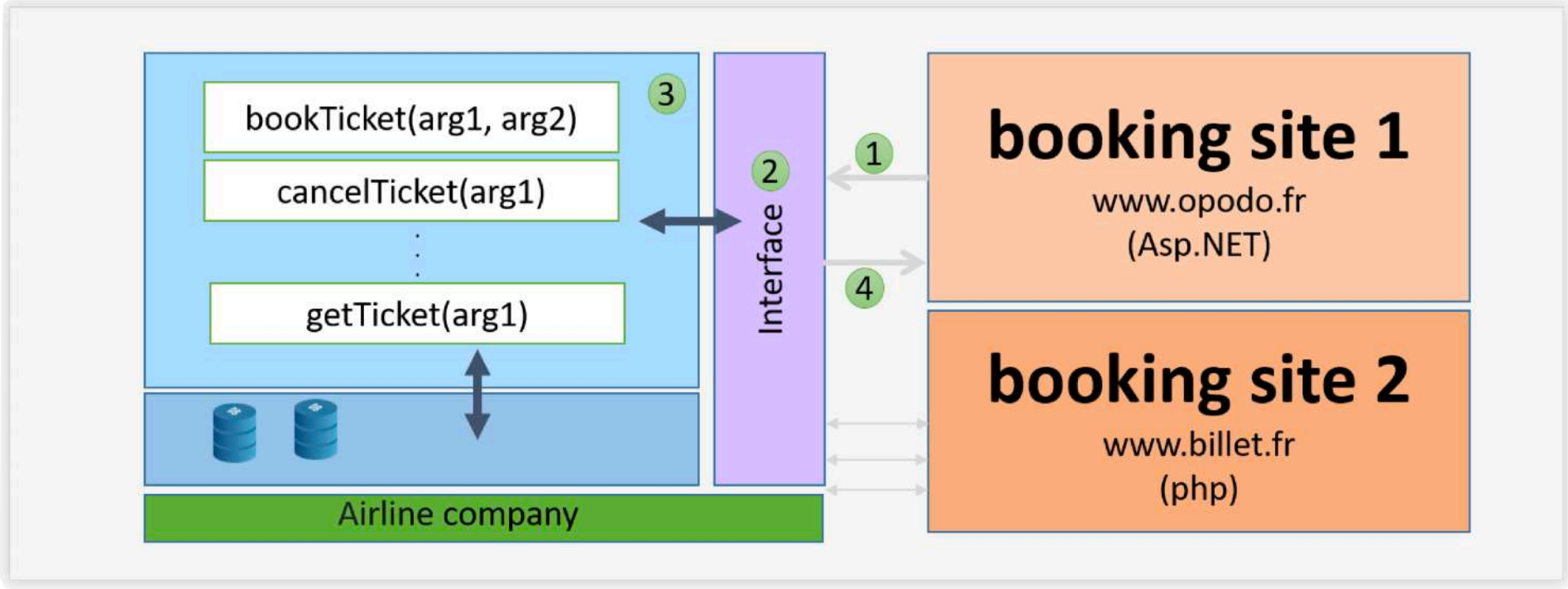
WEB SERVICES - DEFINITIONS

- Web服务是一种软件实体，用于向用户开放功能。
  - 在互联网或内联网上
  - 通过可经由互联网调用的函数
- 基于Web标准
  - HTTP、XML、SOAP、WSDL、UDDI等
- 该功能可跨语言和平台实现
- Web服务是一种独立、模块化、分布式且动态的应用程序



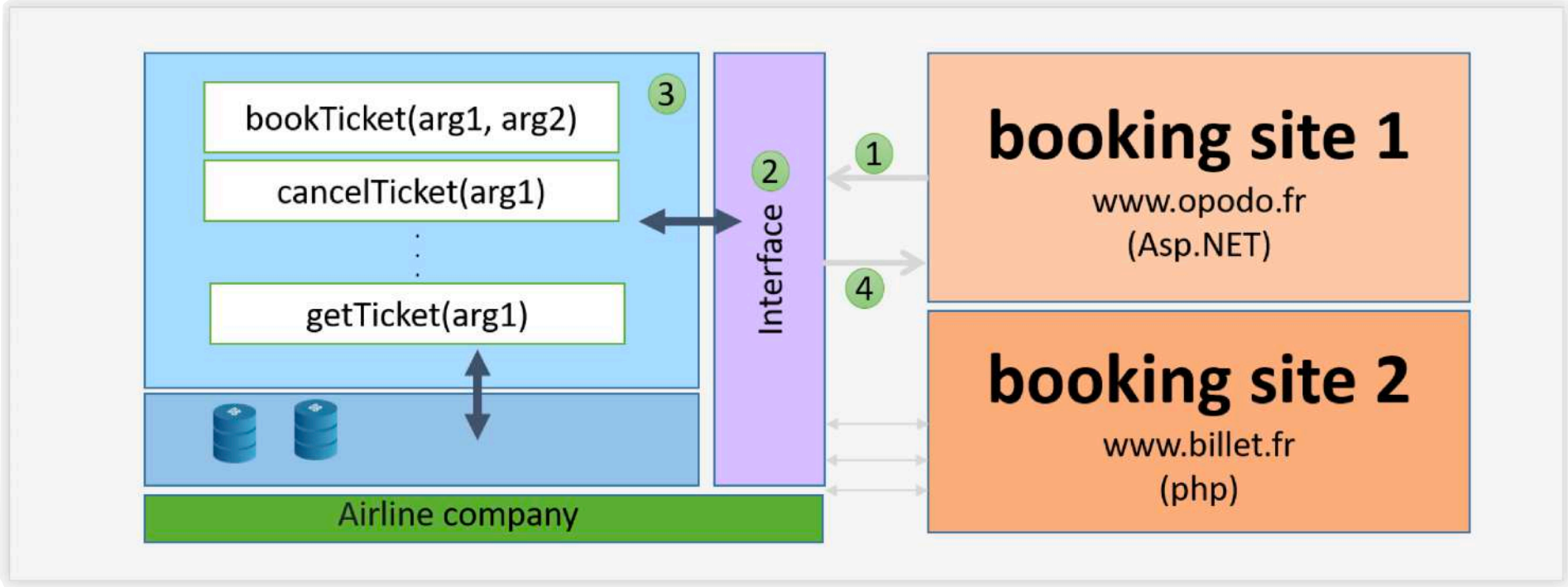
COMMENT ÇA MARCHE ?

- **1 :** Le client envoie une requête au serveur
- **2 :** Le serveur décode la requête et extrait la fonction à appeler
- **3 :** La fonction est exécutée
- **4 :** La réponse est formatée et renvoyé au client



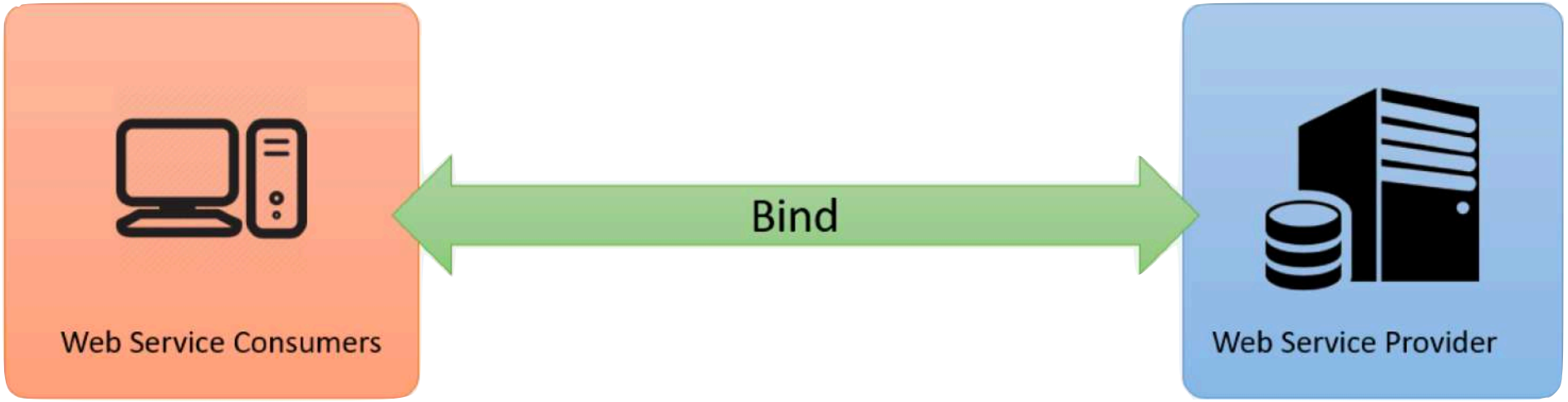
COMMENT ÇA MARCHE ?

- 1 : 客户端向服务器发送请求
- 2 : 服务器解码请求并提取待调用函数
- 3： 函数执行完毕
- 4 : 响应已格式化并返回给客户端



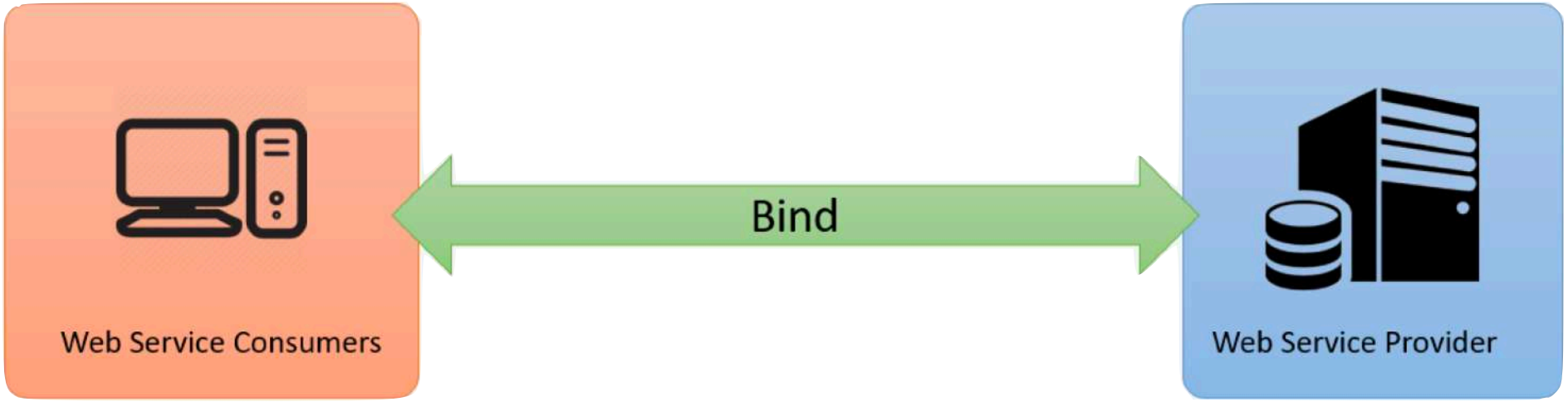
WEB SERVICES - ARCHITECTURE

- Les plus simples Web service ont 2 participants:
  - Le **producteur** (fournisseur), qui présente un interface et une implémentation du service
  - Le **consommateur** (consumer) qui utilise le Web service



WEB SERVICES - ARCHITECTURE

- 最简单的Web服务包含两个参与者：
  - 提供服务接口与实现方案的生产者（供应商）
  - 使用Web服务的消费者



WEB SERVICES - CARACTÉRISTIQUES

- Les Web services utilisent **XML/JSON** pour échanger des messages entre le fournisseur et le consommateur.

- **Exemple d'XML**

```
<employees>
    <employee>
      <firstName>John</firstName> <lastName>Doe</lastName>
    </employee>
    <employee>
      <firstName>Anna</firstName> <lastName>Smith</lastName>
    </employee>
    <employee>
      <firstName>Peter</firstName> <lastName>Jones</lastName>
    </employee>
</employees>
```

- **Exemple de JSON (JSON = JavaScript Object Notation):**

```
{"employees":[
    { "firstName":"John", "lastName":"Doe" },
    { "firstName":"Anna", "lastName":"Smith" },
    { "firstName":"Peter", "lastName":"Jones" }
]}
```

WEB SERVICES - CARACTÉRISTIQUES

- Web服务采用XML/JSON格式在服务提供方与服务使用者之间进行信息交互。

- XML示例

```
< 员工 >
    < 员工>
        <firstname>约翰</<firstname> <last>多</</employee>
    < 员工>
        <firstname>安娜</<firstname>史密斯</<firstname> </employee>
    < 员工>
        <firstname>彼得</<firstname>琼斯</</employee>
</员工>
```

- JSON示例（JSON = JavaScript对象表示法）：

```
{"employees":[
    { "firstName": “John” , “lastName” : “Doe” },
    { "firstName"{ “Anna” , “姓氏” : “Smith” }, {
    { "firstName" “Peter” , “姓氏” : “Jones” }
]}
```

WEB SERVICES - CARACTÉRISTIQUES

- Les Web services utilisent **XML/JSON** pour échanger des messages entre le fournisseur et le consommateur.

- Le producteur du web service s'engage sur les **stabilité des services exposés**, mais **l'implémentation derrière peut évoluer** sans compromettre la capacité du client à interagir avec le service.

- Les services web permettent aux clients d'invoquer des procédures, des fonctions et des méthodes sur des objets distants en utilisant des protocoles standard du web.

WEB SERVICES - CARACTÉRISTIQUES

- **Web服务采用XML/JSON格式在服务提供方与服务使用者之间进行信息交互。**

- 网络服务的生产者承诺保证所提供服务的稳定性，但其底层实现可能持续演进，同时不会影响客户端与服务的交互能力。

- 网络服务使客户能够通过标准网络协议，调用远程对象的程序、函数和方法。

WEB SERVICES - BÉNÉFICES

- **Exposition d'une fonction sur le réseau**
  - Les web services exposent les fonctionnalités d'un code existant sur le réseau.
- **Mise à disposition de données**
  - Les web services permettent de rendre disponible des données avec un protocole uniforme.
- **Interopérabilité**
  - Les web services permettent à diverses applications de communiquer entre elles et de partager des données et des services.
- **Polyvalence**
  - Les web services peuvent être développés sur n'importe quelle plate-forme et dans n'importe quel langage.
- **Flexibilité**
  - Les codes des web services peuvent être réutilisés.
- **Réduction des coûts**
  - Les web services utilisent des normes et des protocoles ouverts pour la communication.

WEB SERVICES - BÉNÉFICES

- 函数在网络上的表达
  - 网络服务通过现有代码在网络中展示功能。
- 数据提供
  - 网络服务能够通过统一协议提供数据访问。
- 互操作性
  - 网络服务使各类应用程序能够相互通信，并共享数据与服务。
- 多价性
  - Web服务可基于任何平台和编程语言进行开发。
- 灵活性
  - Web服务代码可重复使用。
- 降低成本
  - 网络服务采用开放标准和协议进行通信。

2 PRINCIPAUX TYPES DE WEB SERVICES

- **Simple Object Access Protocol (SOAP)**
  - Très lourd, le plus ancien, basé sur des échanges au travers de fichiers XML complexes.
- **REpresentational State Transfer (REST)**
  - Repose sur des URIs et le protocole **HTTP**
  - Utilisation des 5principaux verbes HTTP : POST, HEAD, GET, PUT, DELETE
    - --> proche du concept de CRUD
  - Echange de données au format XML, JSON ou les deux

Create, Read, Update, and Delete (**CRUD**) : les 4 principales actions sur une BD.

CRUD	SQL	HTTP
create	INSERT	PUT
read	SELECT	GET
update	UPDATE	PUT
delete	DELETE	DELETE

2 PRINCIPAUX TYPES DE WEB SERVICES

- 简单对象访问协议
  - 这款最古老的系统运行极其缓慢，其核心依赖于通过复杂XML文件进行数据交换。
- 表征状态转移
  - 基于URI和**HTTP**协议
  - 使用5个主要的HTTP动词：POST、HEAD、GET、PUT，删除
    - --> 接近CRUD概念
  - XML、JSON或两者的数据交换

创建、读取、更新和删除（CRUD）：数据库的四大基本操作。

CRUD	SQL	HTTP
创建	插入	推
读取	选择	获取
更新	更新	推
删除	删除	删除

2 PRINCIPAUX TYPES DE WEB SERVICES

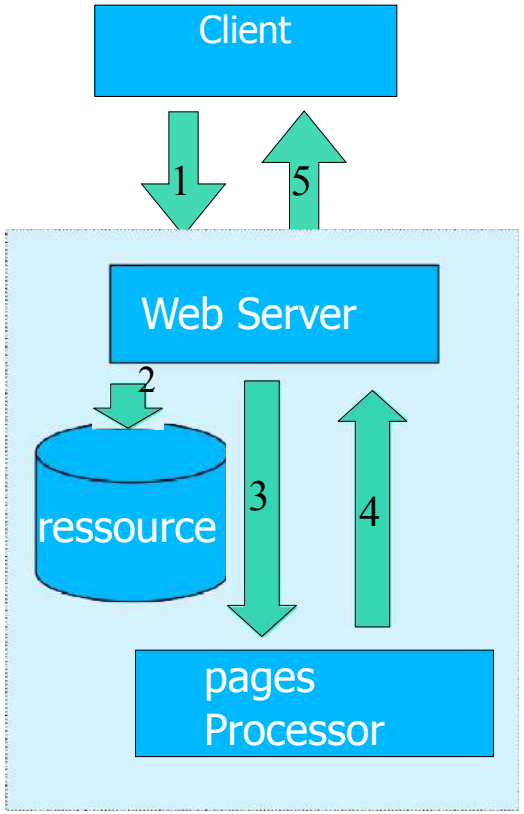
- Nous utiliserons **l'architecture de Web service REST**.
- Nous ajouterons un système de **templating** afin de produire nos pages web dynamiques.

2 PRINCIPAUX TYPES DE WEB SERVICES

- 我们将采用REST架构的Web服务。
- 我们将引入模板系统，用于生成动态网页。

GENERATION D'UN SITE

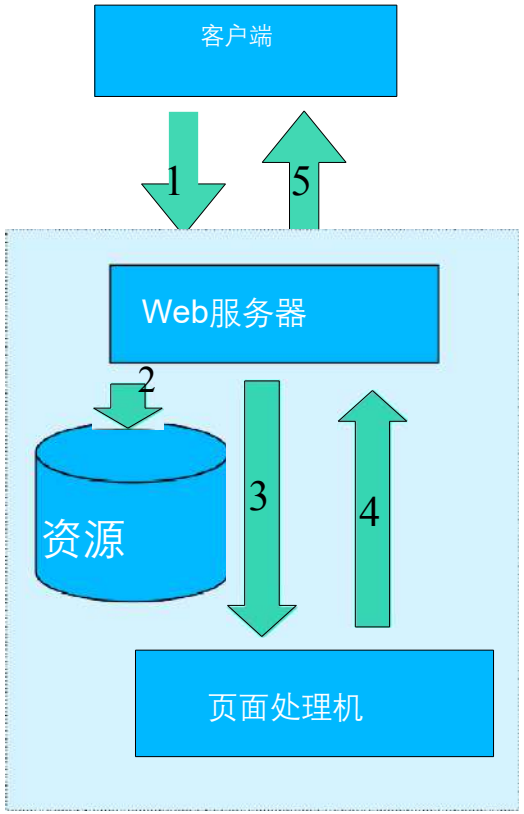
1. Réception d'une requête HTTP
2. Le web service parse la méthode HTTP et l'**achemine** vers la fonction appropriée (**routing**)
3. La fonction récupère les paramètres de la requête, exécute le code, en accédant à des sources de données … et transmet ces données au moteur de rendu de page (*page renderer*) (**templating**)
4. Le moteur génère la page HTML
5. La page générée est renvoyée au consommateur.




```
GET /customers/43456 HTTP/1.1
Host: www.example.org
```

GENERATION D'UN SITE

1. 接收HTTP请求
2. 该网络服务解析HTTP方法，并将其路由至相应功能
3. 该函数负责获取查询参数，通过调用数据源执行代码，并将处理后的数据传递给页面渲染引擎（*page renderer*）（模板化处理）
4. 该引擎生成HTML页面
5. 生成的页面将返回给消费者。




```
GET /customers/43456 HTTP/1.1
Host: www.example.org
```

SPRING FRAMEWORK

Introduction: XML, JSON parsing

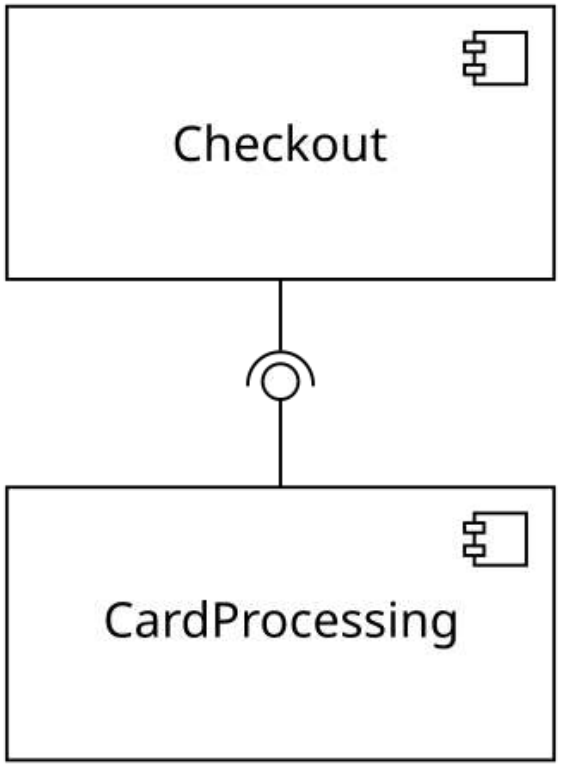
SPRING FRAMEWORK

XML, JSON,

INTRODUCTION AU FRAMEWORK SPRING

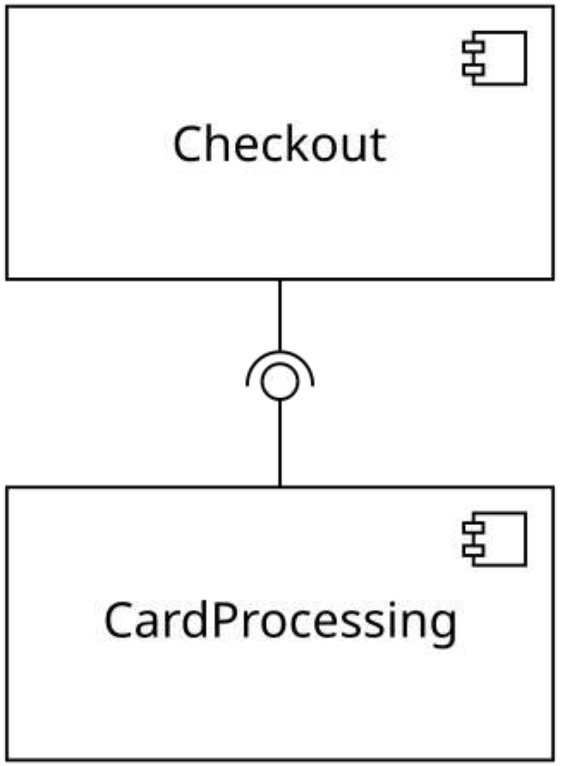
- **Dans le monde Java, ce type de développement se base sur JavaEE et repose sur une approche à base de composants, qui vise**
  - Separation des responsabilités
  - Amélioration de la productivité
  - Indépendence de la plate-forme d'exécution

- Le framework **Spring: vise à fournir les mêmes fonctionnalités dans une approche plus légère**
  - Accès aux données
  - Web
  - Intégration



INTRODUCTION AU FRAMEWORK SPRING

- 在Java领域，这种开发方式以JavaEE为基础，采用基于组件的架构设计，旨在
  - 分权
  - 提高生产力
  - 运行平台的独立性
- Spring框架：旨在以更轻量级的方式提供相同功能
  - 数据存取
  - 网页
  - 整合



LES BUTS AFFICHÉS PAR L'ÉQUIPE DE SPRING GOALS

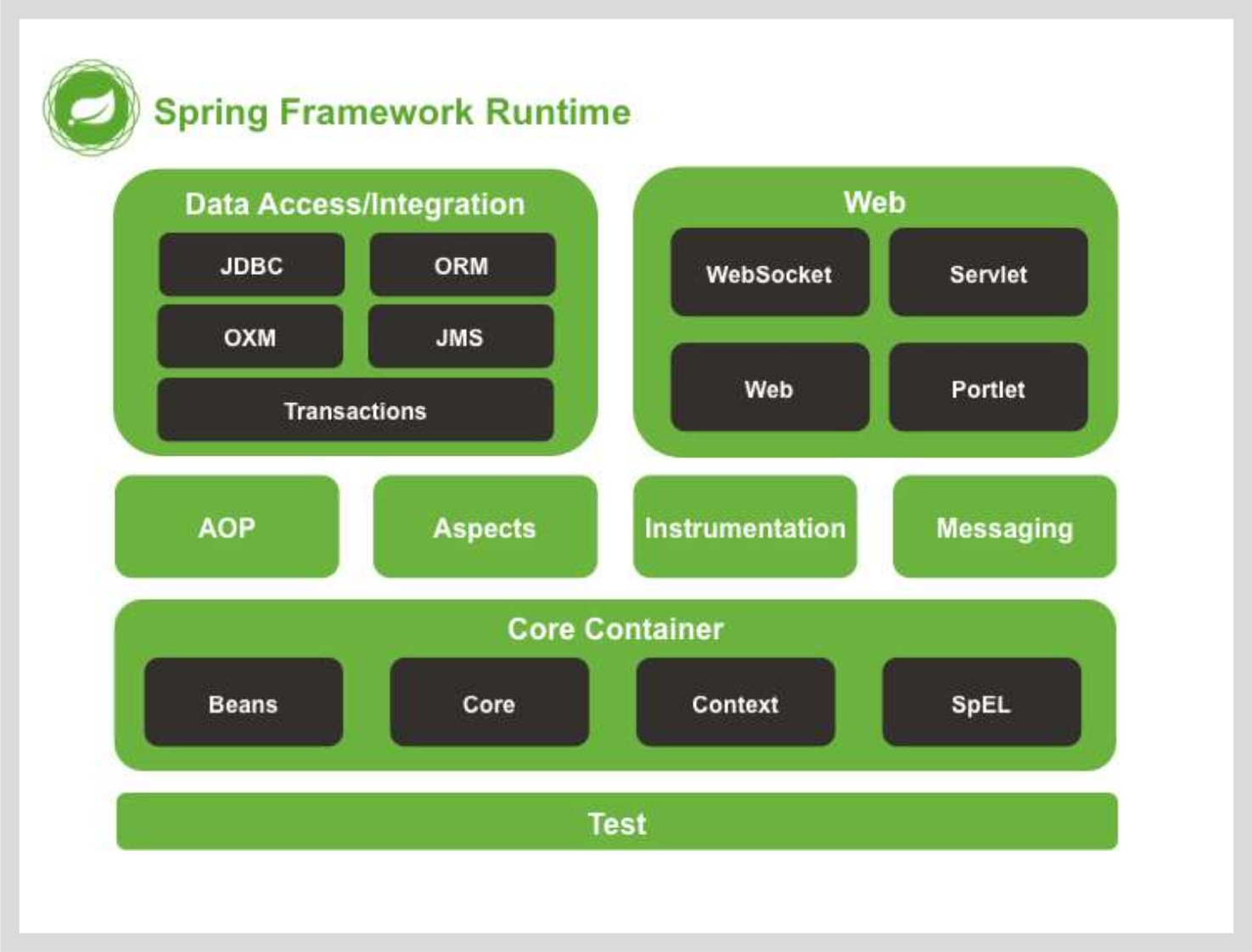
- *J2EE devrait être plus facile à utiliser*
- *La gestion des exceptions est sur-utilisée en Java, un framework ne devrait pas vous forcer à rattraper des exceptions que vous avez peu de chances de pouvoir traiter …*
- *Spring devrait être un plaisir à utiliser*

- ***Spring n'est pas en compétition avec les bonnes solutions/librairies existantes, but devrait les intégrer.*** *(Par exemple, JDO, Toplink, et Hibernate* ***sont de bonnes solutions d'ORM*** *(Object/Relational Mapping))*

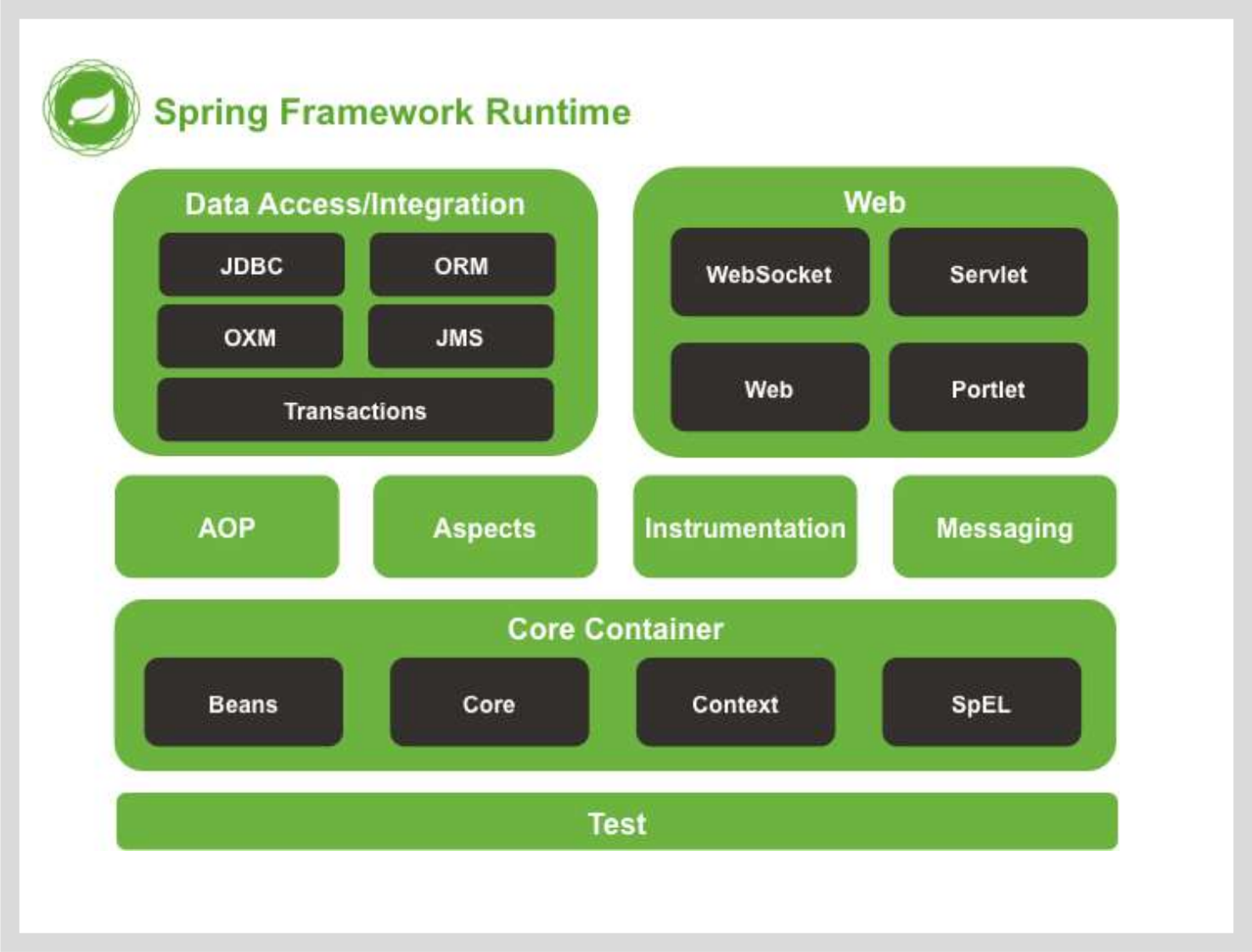
LES BUTS AFFICHÉS PAR L'ÉQUIPE DE SPRING GOALS

- *J2EE应该会更易于使用*
- *Java框架中异常处理机制被过度使用，开发者不该被迫去处理那些自己根本无能为力的异常情况……*
- *Spring应该会是一个非常棒的工具。*
- *Spring并非要与现有优秀的解决方案或库竞争，而是应该将它们整合进来。（例如，JDO、Toplink和Hibernate都是优秀的对象关系映射（ORM）解决方案）*

SPRING FRAMEWORK MODULES OVERVIEW



SPRING FRAMEWORK MODULES OVERVIEW



WEBSERVICES (REST) WITH SPRING

- Spring vous permet également de développer des interfaces graphiques web et des **API REST** :
  - **Routing**
  - Contrôleurs POJO (Plain Old Java Object)
  - Abstraction en ce qui concerne la requête et la réponse HTTP.
  - Sérialisation/désérialisation (par exemple JSON<=>Java)
  - Gestion des exceptions.

  - **Templating** : Couplage avec les vues jsp, velocity, freemarker …

WEBSERVICES (REST) WITH SPRING

- **Spring还支持开发网页图形界面和REST API：**
  - 路由
  - **POJO控制器**
  - **HTTP请求与响应的抽象。**
  - 序列化/反序列化（例如 **JSON<=>Java**）
  - 异常管理。
  - 模板化：与**jsp**视图、**Velocity**、**Freemarker**等工具的集成

SPRINGBOOT

- **Spring Boot** est un moyen rapide de créer une application Spring prête à être mise en production.
  - Il est basé sur le Framework Spring
  - Il est conçu pour vous permettre d'être opérationnel le plus rapidement possible.
  - Il evite les configurations complexes de l'application (souvent faites en XML)

- **SpringBoot = Spring avec une configuration par défaut**

SPRINGBOOT

- Spring Boot是快速构建Spring应用并投入生产的一种便捷方式。
  - 它基于Spring框架
  - 它的设计旨在让您尽快投入实际操作。
  - 它避免了应用程序复杂的配置（通常采用XML格式）
- SpringBoot = Spring默认配置

SPRINGBOOT EN PRATIQUE

Introduction: XML, JSON parsing

SPRINGBOOT EN PRATIQUE

XML , JSON,

APPLICATION PRINCIPALE

- On peut lancer une application SpringBoot avec la simple classe suivante (et quelques dépendances MAVEN):
  cela va exécuter un serveur web, à partir d'une configuration par défaut.

```
@SpringBootApplication
public class Application {
  public static void main(String[] args)     {
      SpringApplication.run(Application.class,args);
   }
}
```

- Cela lance un serveur web accessible à l'adresse http://localhost:8080 (par défaut)

APPLICATION PRINCIPALE

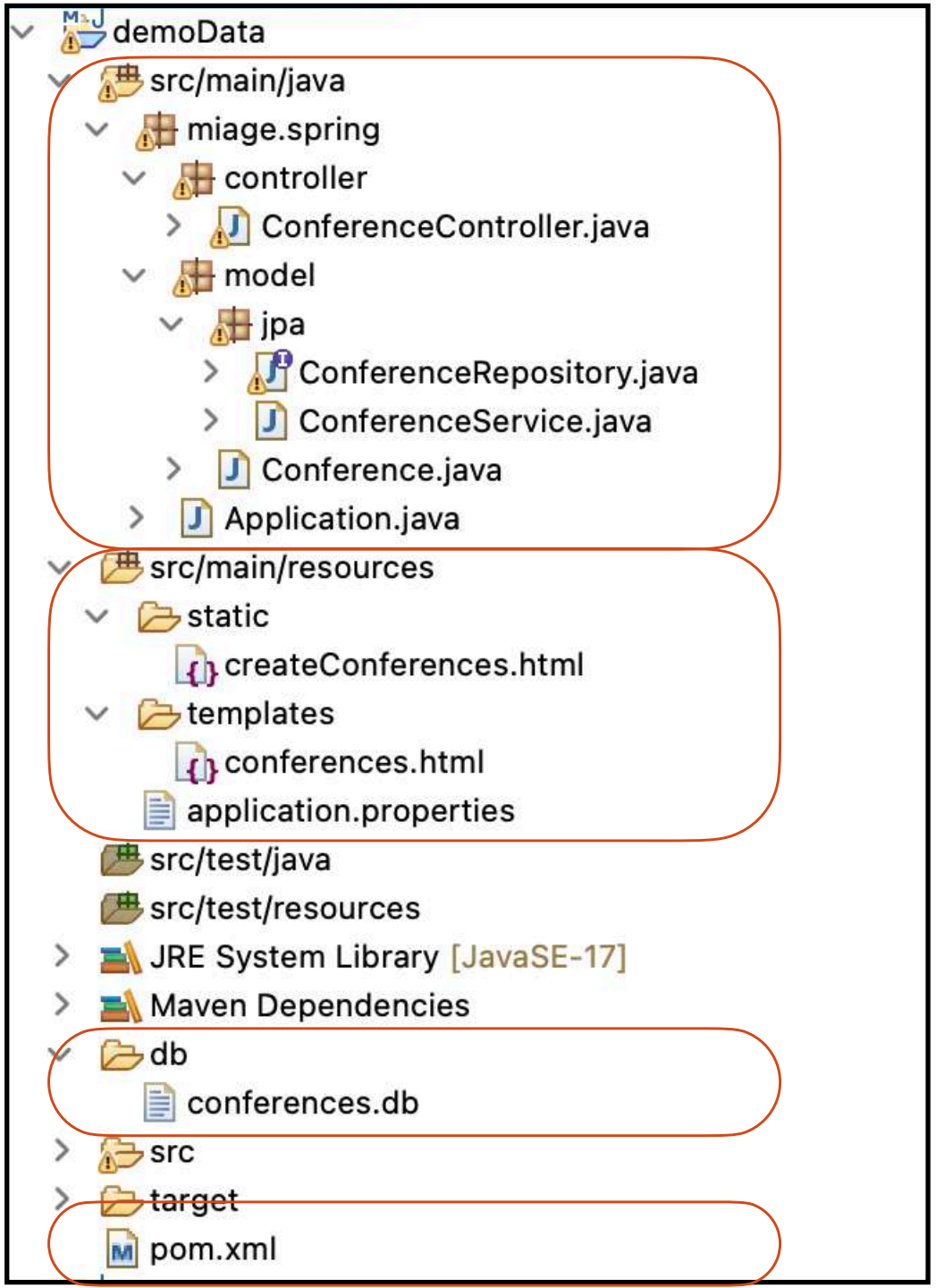
- 只需使用以下类（及少量MAVEN依赖项），即可启动SpringBoot应用：该程序将基于默认配置运行Web服务器。

```
@SpringBootApplication
public class Application {
  public static void main(String[] args)     {
      SpringApplication.run(Application.class,args);
   }
}
```

- 这会启动一个默认地址为http://localhost:8080的网页服务器。

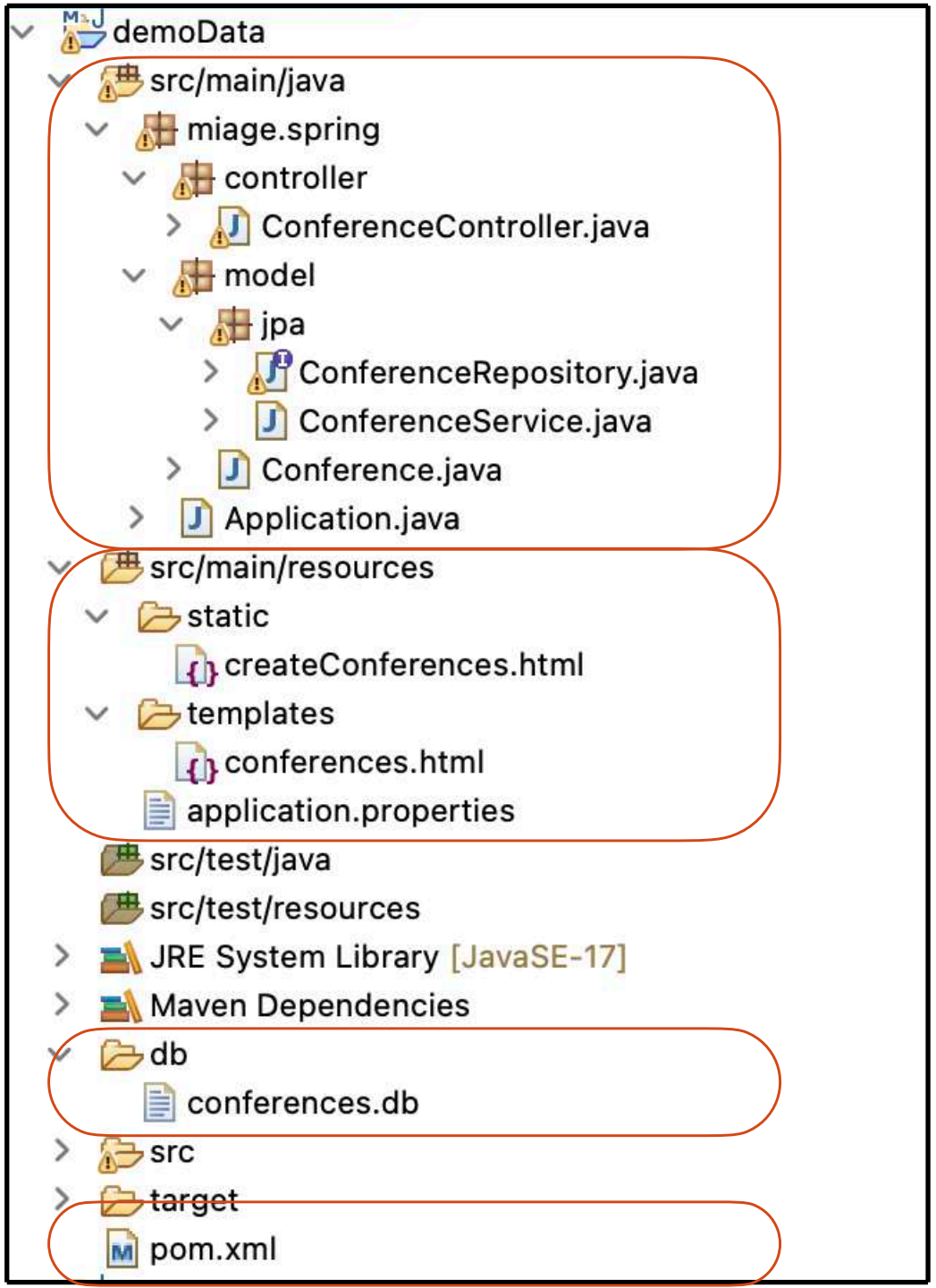
ARCHITECTURE D'UN PROJET SPRINGBOOT

- Un projet SpringBoot contiendra :
  - Le **code Java**
    `src/main/java`
  - Les **ressources** (pages web static, templates, configuration…)
    `src/main/resources`
  - Le fichier **pom.xml** (à la racine)
  - Autres ressources (db…)



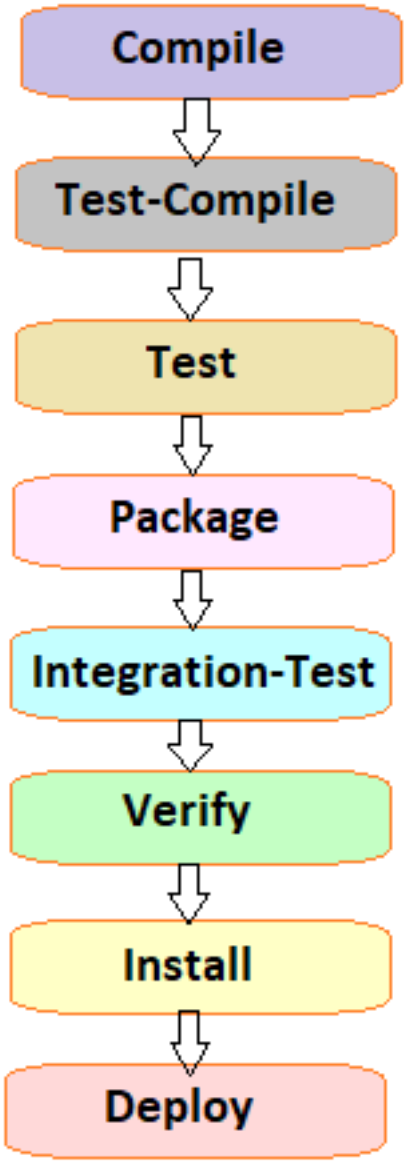
ARCHITECTURE D'UN PROJET SPRINGBOOT

- 一个SpringBoot项目应包含：
  - Java 代码 src/main/java
  - 资源（静态网页、模板、配置等）
    源主资源
  - 根目录下的 pom.xml 文件
  - 其他资源（数据库等）



MAVEN

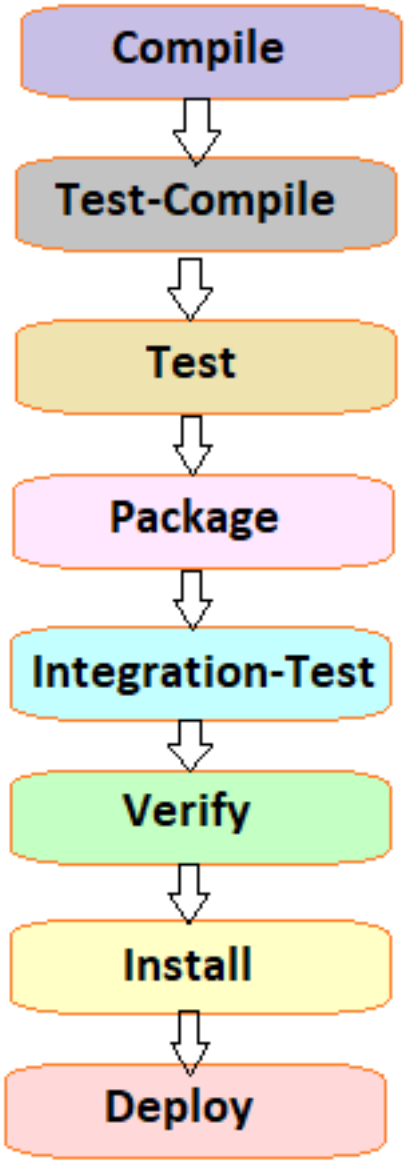
- Maven est un puissant **outil de gestion de projet** qui simplifie le processus de construction des projets basés sur Java tout au long de leur cycle de vie.
- Nous l'utiliserons principalement pour sa **gestion des packages**.



https://geekflare.com/fr/apache-maven-for-beginners/

MAVEN

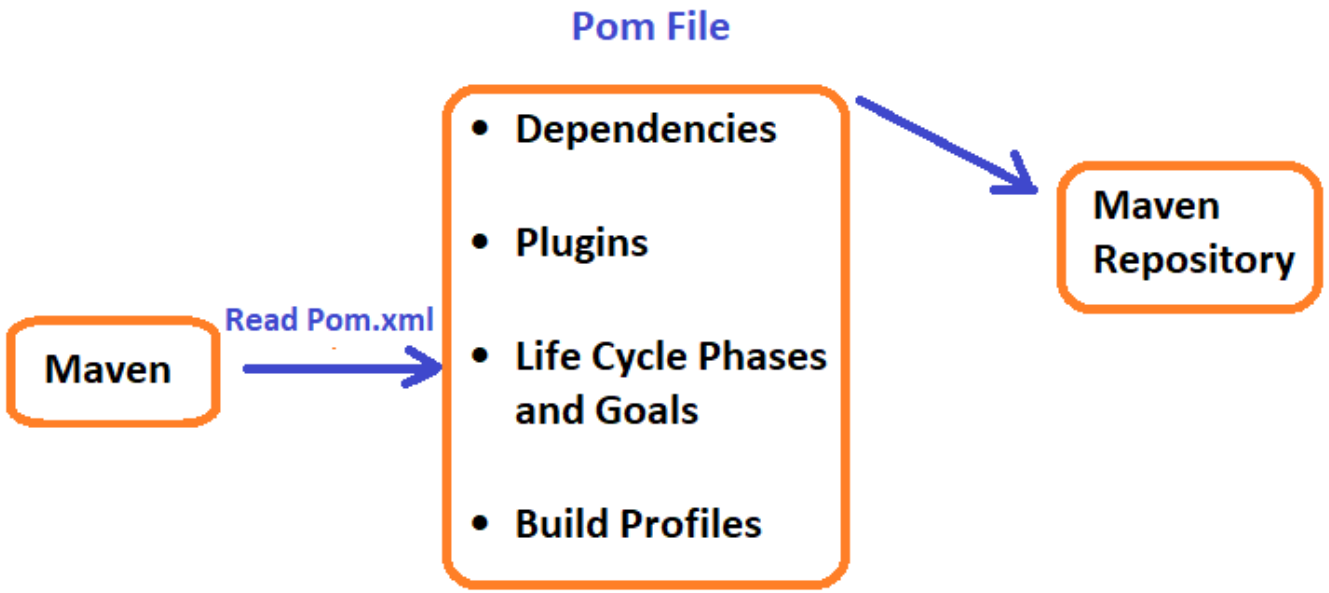
- Maven是一款功能强大的项目管理工具，能够简化Java项目从开发到维护的整个生命周期流程。
- 我们主要会用它来处理包裹。

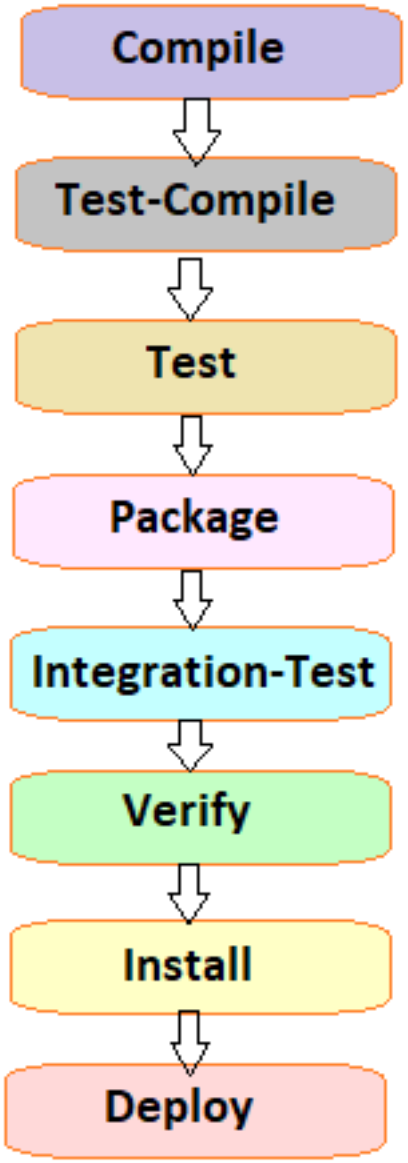


https://geekflare.com/fr/apache-maven-for-beginners/

MAVEN

- Nous l'utiliserons principalement pour sa **gestion des packages**.
  - Tout est déclaré dans le fichier pom.xml.
  - Les fichiers téléchargés sont placés dans le dossier .m2 (dossier caché) de votre ordinateur

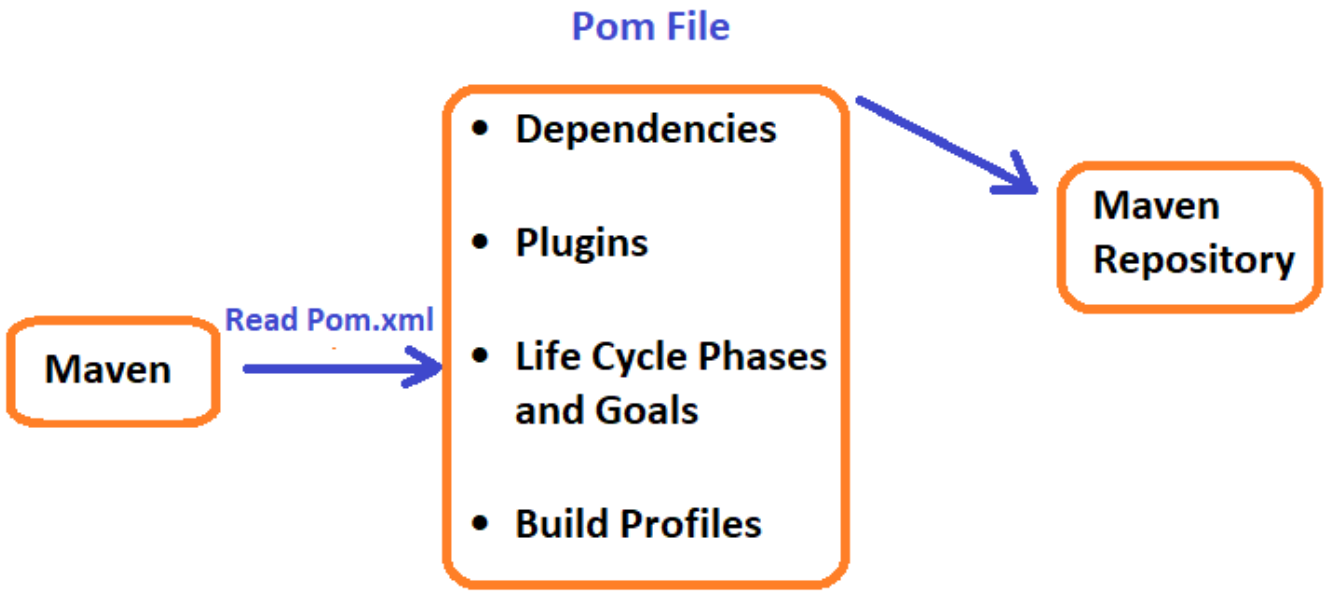


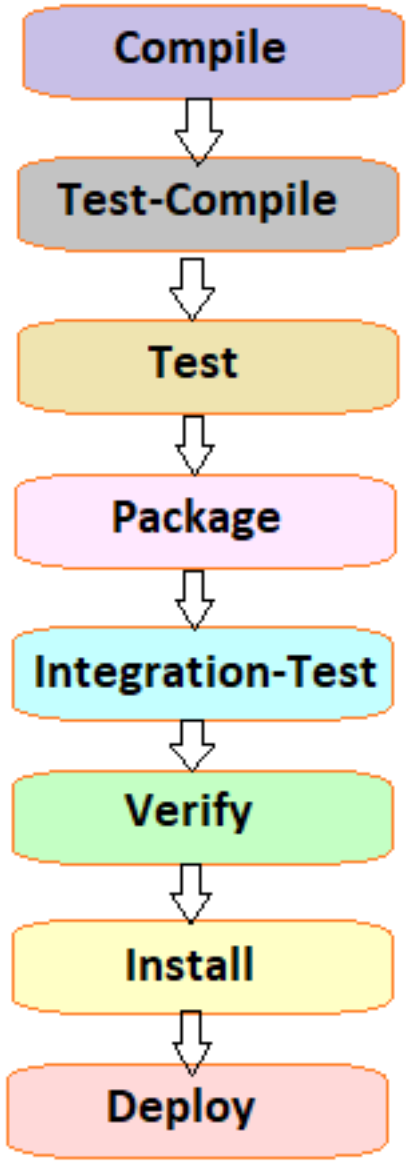


https://geekflare.com/fr/apache-maven-for-beginners/

MAVEN

- 我们主要会用它来处理包裹。
  - 所有内容都已在pom.xml文件中声明。
  - 下载的文件会存入您电脑的.m2（隐藏文件夹）中。





https://geekflare.com/fr/apache-maven-for-beginners/

APPLICATION PRINCIPALE

- Pour que le code fonctionne, vous avez besoin de 2 éléments principaux dans le pom.xml (dans <project>).
  - Avoir comme parent POM Spring Boot starter.

```
<parent>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-parent</artifactId>
      <version>2.4.3</version>
      <relativePath/> <!-- lookup parent from repository -->
</parent>
```

  - Inclure la dependence spring-boot starter web.

```
<dependencies>
   <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
</dependencies>
```

Exemple du fichier pom.xml complet : https://spring.io/guides/gs/spring-boot/

APPLICATION PRINCIPALE

- 要使代码正常运行，您需要在 pom.xml 文件（位于 <project> 目录下）中配置两个关键要素。
  - 父类为 POM Spring Boot 启动器。

```
<parent>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-parent</artifactId>
      <version>2.4.3</version>
      <relativePath/> <!-- lookup parent from repository -->
</parent>
```

  - 包含Spring Boot Web启动器的依赖项。

```
<dependencies>
   <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
</dependencies>
```

完整 pom.xml 文件示例： https://spring.io/guides/gs/spring-boot/

RAPPEL: JAVA ANNOTATIONS

RAPPEL: JAVA ANNOTATIONS

XML, JSON,

JAVA ANNOTATIONS

- Introduction officielle Oracle Java Doc

- *Les annotations sont une forme de **métadonnées**, qui fournissent des données sur un programme mais qui ne font pas partie du programme lui-même. Les annotations n'ont pas d'effet direct sur le fonctionnement du code qu'elles annotent.*
- *Les annotations ont un certain nombre d'utilisations, parmi lesquelles*
  - ***Informations pour le compilateur** - Les annotations peuvent être utilisées par le compilateur pour détecter des erreurs ou supprimer des avertissements.*
  - ***Traitement au moment de la compilation et du déploiement** - Les outils logiciels peuvent traiter les informations d'annotation pour générer du code, des fichiers XML, etc.*
  - ***Traitement au moment de l'exécution** - Certaines annotations peuvent être examinées au moment de l'exécution.*

JAVA ANNOTATIONS

- 官方介绍Oracle Java Doc

- *注释属于元数据的一种形式，用于提供程序相关信息，但本身并不属于程序组成部分。注释不会直接影响被注释代码的运行效果。*
- *注释具有多种用途，其中包括*
  - *编译器信息 - 编译器可利用注释来检测错误或消除警告。*
  - *编译与部署时的处理——软件工具可处理注释信息，生成代码、XML文件等。*
  - *执行时处理 - 部分注释可在执行时进行检查。*

JAVA ANNOTATIONS

- Pour le compilateur :

```
@Override
void mySuperMethod() { ... }
```

- Pour le compilateur, avec paramètres :

```
@SuppressWarnings(value = "unchecked")
void myMethod() { ... }
```

JAVA ANNOTATIONS

- 关于编译器：

```
重叠
void mySuperMethod() { ... }
```

- 针对编译器，参数设置如下：

```
@SuppressWarnings(value = “unchecked” ) void myMethod() { ... }
```

JAVA ANNOTATIONS

- Annotations standards Java (JSR250, JSR 330, JSRxxx, …

```
@PostContruct
void myDestroyMethod() { ... )
```

```
@Entity  // indicate that this class is an entity to be managed by JPA
public class myEntity { ... }
```

- Junit exemples :

```
@BeforeMethod // indicate that this method should be run before the tests void
myInitMethod() { ... }
```

```
@Test
public void mySuperTest() { ... }
```

- Spring exemples :

```
@Component
public class mySuperComponent { ... }
```

JAVA ANNOTATIONS

- 注释标准 Java（JSR250、JSR330、JSRxxx等）

```
@后构造
void myDestroyMethod() { ... )
```

```
// indicate that this class is an entity to be managed by JPA公共类myEntity{ ...}
```

- 六、示例：

```
BeforeMethod// 指示此方法应在测试前运行void myInitMethod() { ... }
```

```
测试
public void mySuperTest() { ... }
```

- 春季示例：

```
组件
public class mySuperComponent { ... }
```

ANNOTATION: SYNTAXE

- Une annotation peut avoir 1 ou plusieurs paramètres :

```
@RequestParam("name")
@RequestParam(value = "name", defaultValue = "test")
```

- Avec 1 paramètre, seule la valeur est nécessaire.
- Avec plusieurs paramètres, **la clé « valeur » est nécessaire**

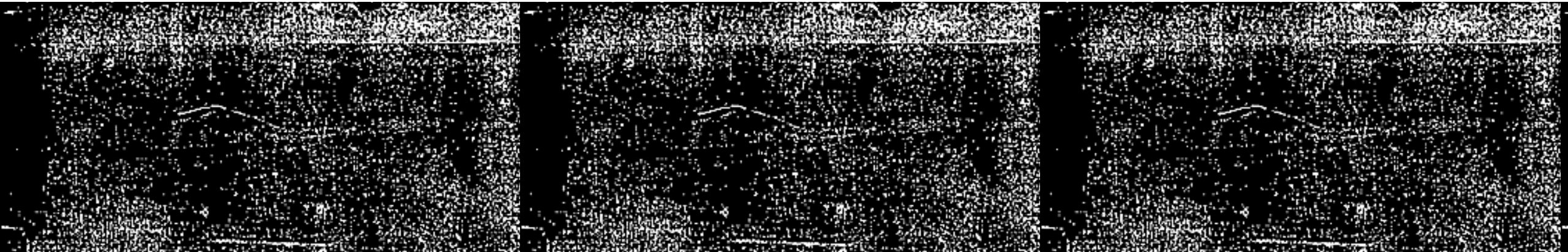
ANNOTATION: SYNTAXE

- 一个注释可以包含一个或多个参数：

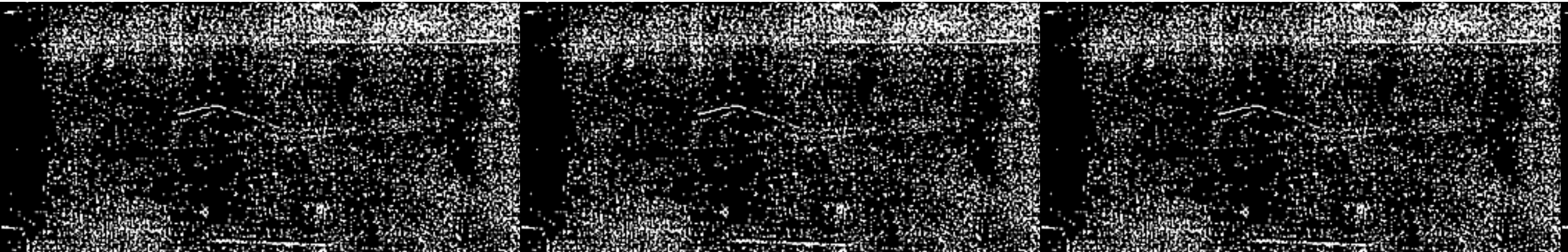
```
人参（“名”）
@RequestParam(value = “name”, defaultValue = “test”)
```

- 当参数为1时，仅需提供数值即可。
- 当涉及多个参数时，必须使用“值”键

ROUTING



ROUTING



REQUÊTE HTTP

- **Request Line:**
  - Contient la méthode (GET, POST, PUT…), le chemin vers la ressource, et la version d' HTTP.
  - Exemple: `GET /index.html HTTP/1.1`
- **Request Headers:**
  - Fournit des metadata sur la requête.

```
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
```

- **Request Body (optional):**
  - Utilisé pour envoyé des données.
  - Exemple d'une requête POST :

```
username=john&password=1234
```

REQUÊTE HTTP

- 请求行：
  - 包含方法（GET、POST、PUT等）、资源路径及版本号。HTTP。
  - 示例：`GET /index.html HTTP/1.1`
- 请求头：
  - 提供查询的元数据。

```
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
```

- 请求体（可选）：
  - 用于发送数据。
  - POST请求示例：

```
username=john&password=1234
```

ROUTING: DÉFINITION D'UN CONTRÔLEUR

- Toutes les méthodes à associer à des routes doivent être implémentés dans une **classe annotée par une annotation de contrôleur** :
  - `@RestController`
    Pour les contrôleurs, renvoyant des données (textes, XML, JSON…)
  - `@Controller`
    Pour les controllers renvoyant des pages web (fichiers HTML ou construction de page HTML avec un moteur de rendu).

```
@RestController
public class MyControler {

   @RequestMapping("/")
   public String welcome() {
      return "Hello World";
   }
}
```

ROUTING: DÉFINITION D'UN CONTRÔLEUR

- 所有需与路由关联的方法都必须实现
  在带有控制器注释的类中：
  - 控制台
    用于控制器的返回数据（文本、XML、JSON等）
  - 控制台
    适用于返回网页的控制器（包括HTML文件或通过渲染引擎生成的HTML页面）。

```
控制台
public class MyControler {

    @RequestMapping("/")
    public String welcome() {

        返回 "Hello World" ; }
}
```

ROUTING

Serveur lancé sur http://localhost:8080

- Ensuite les méthodes ou la classe vont être annotée pour spécifier quelle requête elles capturent.
- `@RequestMapping("/")` sur une méthode va lier l'appel de cette méthode sur l'adresse **localhost:8080/**
- `@RequestMapping("/jean")` sur une méthode va lier l'appel de cette méthode sur l'adresse **localhost:8080/jean**
- `@RequestMapping("/jean")` est similaire à `@GetMapping("/jean")`
- D'autres annotations existent `@PostMapping, @PutMapping` …

ROUTING

服务器启动于http://localhost:8080

- 随后，该方法或类将被添加注释，以明确其捕获的查询类型。
- RequestMapping（“/”）方法会将该方法的调用链接到本地主机8080/
- 请求映射 “/jean” 到方法时，会将该方法的调用地址绑定到 localhost:8080/jean
- @RequestMapping(“/jean”)与@GetMapping(“/jean”)类似
- 其他注释包括@PostMapping、@PutMapping等

ROUTING

GET **localhost:8080/**

GET **localhost:8080/conferences/**

GET **localhost:8080/conferences/first/**

```
@Controller
public class ConferenceController {

    @GetMapping("/")
    public String getConferences() {
        return "conferences";
    }

    @GetMapping("/conferences")
    public String addUserToConference() {
        return "conferences";
    }

    @GetMapping("/conferences/first")
    public String getAllConferences1() {
        return "First conferences";
    }
}
```

ROUTING

获取本地主机:8080/

获取本地主机:8080/会议/

获取本地主机：8080/会议/首场/

```
控制台
public class ConferenceController {

    @GetMapping("/")
    public String getConferences() {
        return      会议      ;
    }

    @GetMapping( "/conferences" )
    public String addUserToConference() {
        return "conferences";
    }

    @GetMapping( "/conferences/first" )public String
    getAllConferences1() {return " First
    conferences" ;
    }
}
```

ROUTING : FACTORISATION

GET **localhost:8080/users/**

GET **localhost:8080/users/first/**

```
@Controller
@RequestMapping("/users")
public class UserController {

    @GetMapping("/")
    public String getUsers() {
        return "users";
    }

    @GetMapping("/first")
    public String getFirstUser() {
        return "first user";
    }
}
```

ROUTING : FACTORISATION

获取本地主机:8080/用户/

获取本地主机:8080/用户/首/

```
@Controller @RequestMapping( "/
users" )public class UserController {

    获取 "/"
    public String getUsers() {
        return 用户    ;
    }

    获取 "<第一"
    public String getFirstUser() {
        return "first user";
    }
}
```

ROUTING

GET **localhost:8080/users/**

GET **localhost:8080/users/first/**

POST **localhost:8080/users/first/**

```
@RestController
@RequestMapping("/users")
public class UserControleur {

    @GetMapping("/")
    public String getUsers() {
       return "users";
   }

    @GetMapping("/first")
    public String getFirstUser() {
       return "first user";
   }

   @PostMapping("/first")
    public String postFirstUser() {
       return "POST first user";
   }
}
```

ROUTING

获取本地主机:8080/用户/

获取本地主机:8080/用户/首/

POST **localhost:8080/users/first/**

```
控制台
@RequestMapping（“/用户”）公共类

UserControleur{@GetMapping（“/”）
    public String getUsers() {
        return    用户   ;
    }

    获取 “<第一”
    public String getFirstUser() {
        return "first user";
    }

    发帖后“第一”
    public String postFirstUser() {
        return "POST first user";
    }
}
```

PATHVARIABLE

- Pour **relier 1 méthode à plusieurs URL**, on peut utiliser une `PathVariable`:

```
@RequestMapping("/hello/{name}")
public String welcome(@PathVariable String name) {
    return "Hello " + name;
}
```

- cf. PathVariable in https://www.baeldung.com/spring-requestmapping

PATHVARIABLE

- 若需将1种方法关联至多个URL，可使用PathVariable：

```
@RequestMapping ( “/hello/{name}” )
public String welcome(@PathVariable String name) {
    return "Hello " + name;
}
```

- 参见https://www.baeldung.com/spring-requestmapping中的PathVariable

ROUTING : PATHVARIABLE

GET **localhost:8080/users/**

GET **localhost:8080/users/first/**

GET **localhost:8080/users/number/first**

GET **localhost:8080/users/number/second**

GET **localhost:8080/users/number/third**

```
@Controller
@RequestMapping("/users")
public class UserController {

    @GetMapping("/")
    public String getUsers() {
        return "users";
    }

    @GetMapping("/first")
    public String getFirstUser() {
        return "first user";
    }

    @GetMapping("/number/{nb}")
    public String getNbUser(@PathVariable
String nb) {
        return nb + " user";
    }
}
```

ROUTING : PATHVARIABLE

获取本地主机:8080/用户/

获取本地主机:8080/用户/首/

获取本地主机:8080/用户/编号/第一、本地主机:8080/用户/编号/第二、本地主机:8080/用户/编号/第三

```
控制台
@RequestMapping( “/users” )
public class UserController {

    @GetMapping("/")
    public String getUsers() {
        return 用户    ;
    }

    获取 “<第一”
    public String getFirstUser() {
        return "first user";
    }

    获取 “number/{nb}”
    public String getNbUser(@PathVariable
String nb) {

        return nb+ “用户” ; }
}
```

ROUTING : REQUESTPARAM

- Gérer les URLs avec paramètres :
  http://localhost:8080/hello?name=Jean

```
@RequestMapping("/hello")
public String welcomeName(@RequestParam String name) {
    return "Hello " + givenName + " " + name;
}
```

- **Autant de RequestParam que nécessaires peuvent être utilisés.**
  Avec une méthode **POST**, les paramètres sont passés dans le body du protocole HTTP et sont gérés de la même manière.

- Ils peuvent être combinés avec @PathVariable

ROUTING : REQUESTPARAM

- 管理带参数的URL：http://localhost:8080/hel-lo？name=Jean

```
@RequestMapping ( “/hello” )
public String welcomeName(@RequestParam String name) {
    return "Hello " + givenName + " " + name;
}
```

- 可以使用任意数量的RequestParam参数。在POST方法中，这些参数会被传递到HTTP协议的body部分，并采用相同的方式进行处理。

- 它们可以与 @PathVariable 结合使用。

ROUTING : REQUESTPARAM

- **http://localhost:8080/hello?name=Jean**
- Le nom du paramètre de la méthode welcomeName doit être le même que celui dans l'URL.

```
@RequestMapping("/hello")
public String welcomeName(@RequestParam String name) {
    return "Hello " + name;
}
```

- Variantes : si on souhaite que le nom de la variable Java soit différente.

```
@RequestMapping("/hello")
public String welcomeName(@RequestParam("name") String n) {
    return "Hello " + n;
}
```

- Plus d'options :

```
@RequestParam(value = "name", defaultValue = "test")
```

ROUTING : REQUESTPARAM

- **http://localhost:8080/hello？name=Jean**
- 欢迎名称方法的参数名称必须与URL中的那个。

```
 "/hello"
public String welcomeName(@RequestParam String name) {
    return "Hello " + name;
}
```

- 变体：若需使Java变量名称不同。

```
@RequestMapping ( "/hello" )
public String welcomeName(@RequestParam("name") String n) {
    return "Hello " + n;
}
```

- 更多选项：

```
@RequestParam(value = "name" , defaultValue = "test" )
```